Student Management System Documentation (Python)

1. Introduction

This project is a **basic Student Management System developed in Python** that allows users to manage student information through a console interface.

The system uses **Python data structures such as lists and dictionaries**, along with **functions and control structures**, to perform the following operations:

- Register new students
- Display the student list
- Search for students
- Update student information
- Delete students

The main purpose of this project is to **practice fundamental programming concepts in Python**, including data structures, functions, loops, and conditional statements.

---

2. Data Structure

The program stores student information using a **list of dictionaries**.

```
registros = [
    {"id": 1143245632, "nombre": "Randy", "edad": 19, "programa":
"sistemas", "estado": "activo"}
]
```

Explanation

- **List (`[]`)**

  A list is used to store multiple student records.

- **Dictionary (`{}`)**

  Each student is represented as a dictionary containing their information.

Student Fields

Field	Description
id	Student identification number
nombre	Student name
edad	Student age
programa	Academic program
estado	Student status (active or inactive)

Example of a student record

```
{
"id": 1143245632,
"nombre": "Randy",
"edad": 19,
"programa": "sistemas",
"estado": "activo"
}
```

---

3. System Functions

The program is organized using **functions**, which help structure the code and make it reusable.

3.1 Function: `registrar_estudiantes()`

Purpose

This function **registers a new student in the system**.

Code

```
def registrar_estudiantes(estudiantes):

    estudiantes = {
        "id": id,
        "nombre": nombre,
        "edad": edad,
        "programa": programa,
        "estado": estado
    }

    registros.append(estudiantes)

    print("Successful Student Registration")
```

How it works

1. A **dictionary is created** with the student's information.
2. The dictionary is added to the **registros list** using the `append()` method.
3. A confirmation message is displayed.

Method used

```
append()
```

The `append()` method **adds a new element to the end of a list**.

Example:

```
lista.append(new_element)
```

3.2 Function: `consultar_lista_estudiantes()`

Purpose

This function **displays all students stored in the system**.

Code

```
def consultar_lista_estudiantes(registros):

    if len(registros) == 0:
        print("The list is empty")
        return

    for e in registros:
        print(f"Id: {e['id']} | Nombre: {e['nombre']} | Edad: {e['edad']}
| Programa: {e['programa']} | Estado: {e['estado']}")
```

How it works

1. The function first checks if the list is empty using:

```
len(registros)
```

2. If the list is empty, a message is displayed.
3. If the list contains students, a **for loop** is used to iterate through each student and display their information.

Loop used

```
for e in registros
```

This loop iterates through **each student stored in the list**.

3.3 Function: `buscar_estudiantes()`

Purpose

This function **searches for a student by name**.

Code

```
def buscar_estudiantes(registros, nombre):

    for e in registros:
        if e["nombre"].lower() == nombre.lower():
            return e

    return None
```

How it works

1. The function loops through all students in the list.
2. It compares the entered name with the stored student name.
3. The `.lower()` method is used to avoid issues with uppercase or lowercase letters.
4. If the student is found, the function returns the student's dictionary.
5. If the student is not found, it returns **None**.

Example usage:

```
resultado = buscar_estudiantes(registros, "Randy")
```

3.4 Function: `actualizar_informacion()`

Purpose

This function **updates a student's age or status**.

Code

```
def actualizar_informacion(registros, nombre, nueva_edad, nuevo_estado):
```

```
    estudiantes = buscar_estudiantes(registros, nombre)

    if estudiantes is None:
        print("Student not found")
        return

    if nueva_edad is not None:
        estudiantes["edad"] = nueva_edad

    if nuevo_estado is not None:
        estudiantes["estado"] = nuevo_estado

    print("Student updated successfully")
```

How it works

1. The function searches for the student using the `buscar_estudiantes()` function.
2. If the student does not exist, a message is displayed.
3. If the student exists:
    - The age is updated if a new value is provided.
    - The status is updated if a new value is provided.

Example:

```
estudiantes["edad"] = nueva_edad
```

3.5 Function: `eliminar_estudiante()`

Purpose

This function **removes a student from the system**.

Code

```
def eliminar_estudiante(registros, nombre):

    estudiantes = buscar_estudiantes(registros, nombre)

    if estudiantes:
        registros.remove(estudiantes)
        print("Student delete")
    else:
        print("Student not found")
```

How it works

1. The function searches for the student.
2. If the student exists, it removes the student from the list using `remove()`.

3. If the student does not exist, a message is displayed.

Method used

```
remove()
```

The `remove()` method **removes a specific element from a list**.

---

4. Main Menu System

The program uses an **infinite loop (`while True`)** to continuously display the menu.

```
while True:
```

Menu options

```
print("\n--- STUDENT MANAGEMENT ---")
print("1. Register new students")
print("2. Check the student list")
print("3. Find a student")
print("4. Update student information")
print("5. Delete students")
```

Each option allows the user to perform a specific operation in the system.

---

5. User Input Control

The system uses **if / elif conditional statements** to execute the selected option.

Example:

```
if opcion == "1":
```

Menu actions

Option	Action
1	Register a new student
2	Display student list
3	Search for a student
4	Update student information

Option	Action
5	Delete a student

---

6. Error Handling

The program uses **try / except blocks** to prevent the program from crashing due to invalid input.

Example:

```
try:
    edad = int(edad)
except ValueError:
    print("Invalid data")
```

This prevents errors if the user enters non-numeric data where a number is expected.

---

7. Programming Concepts Used

This system demonstrates several **fundamental Python programming concepts**:

- Lists
- Dictionaries
- Functions
- Conditional statements (`if`)
- Loops (`for`, `while`)
- Error handling (`try / except`)
- List methods

---

8. Possible Improvements

The system can be improved by adding more advanced features such as:

- ID duplication validation
- Search by ID instead of name
- Data persistence using CSV files
- Graphical user interface (GUI) with Tkinter
- Database integration using SQLite

9. Conclusion

This project demonstrates how **basic programming concepts in Python** can be used to build a simple **Student Management System**.

The program allows users to:

- Manage student information
- Practice data manipulation
- Apply logical programming structures

Projects like this are important for **developing programming logic and software development skills**, especially for beginners learning Python.